

Model	Tri-Sq	Tri-Pen	Tri-Hex	Tri-Hep	Tri-Oct	Sq-Pen	Sq-Hep	Irr-Poly	Arrow	Star	Plus	Arrow-Plus
LLaVA-1.5	0%	0%	10%	30%	0%	0%	0%	10%	0%	0%	0%	0%
LLaVA-Next	0%	0%	10%	0%	0%	0%	0%	10%	0%	0%	0%	0%
LLaVA-OneVision	0%	0%	100%	0%	0%	0%	0%	36%	0%	0%	0%	0%
Qwen2-VL	0%	20%	40%	40%	0%	0%	0%	19%	0%	0%	0%	0%
InternVL	10%	0%	0%	0%	0%	0%	0%	18%	0%	0%	0%	0%
LLaMA-3.2	90%	0%	40%	0%	0%	0%	0%	36%	0%	0%	0%	0%
Molmo	40%	40%	70%	50%	0%	10%	0%	60%	42%	0%	3%	0%
Janus-Pro	70%	0%	70%	0%	0%	0%	0%	35%	0%	0%	0%	0%
Math-LLaVA	0%	0%	0%	0%	0%	0%	0%	42%	0%	0%	0%	0%
G-LLaVA	0%	0%	40%	44%	10%	0%	0%	100%	23%	0%	0%	0%
Math-PUMA	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%
GPT-4-Turbo	50%	0%	30%	0%	10%	0%	10%	38%	89%	34%	92%	0%
GPT-4o	60%	0%	60%	0%	60%	0%	10%	42%	89%	84%	88%	0%

Table 5: **Performance on merged shapes, irregular polygons, and abstract shapes.** (see examples in Figure 4). GPT-4o and GPT-4-Turbo excel on common shapes ('arrow,' 'star,' 'plus') but fail (0%) on unseen shapes, suggesting reliance on recognition over side counting.

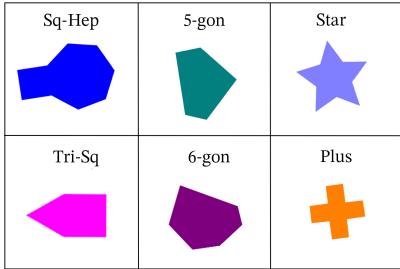


Figure 4: Examples of abstract shapes. For the full set of shapes, see Figure B.3

ing—memorizing shape-side relationships rather than actively counting and using visual information. These shapes are unlikely to appear in training, allowing us to evaluate whether models can generalize beyond familiar regular polygons. We consider three distinct categories of abstract shapes:

Merged Shapes: We combine two regular polygons, such as a triangle and a square, where the shapes share a side. A simple shared-side algorithm can be applied to obtain the number of the sides: $s_1 + s_2 - 2$ where s_1 and s_2 are the number of sides of shape 1 and shape 2, respectively. Applying this algorithm to a triangle and a square gives: $3 + 4 - 2 = 5$ total sides. Table 5 shows that models generally fail to implement this algorithm, with most achieving 0% accuracy on merged shapes (Tri-Sq through Sq-Hep). LLaVA-OneVision and LLaMA-3.2 each achieve over 90% accuracy on exactly one merged shape. This is because most open-source models (except for Molmo, which is trained on counting) tend to guess a single value from 3 to 8 on every shape, thus getting “guessing correctly” on a specific category while scoring 0% on all others. These results demonstrate that mod-

els do not implement the $s_1 + s_2 - 2$ algorithm.

Irregular Polygons: Extending the experiment from Section 3.1, we evaluate side-counting on irregular polygons with 3 to 8 sides. Performance remains low across all models, with GPT-4o, Molmo, and Math-LLaVA achieving the highest accuracy (42%). Most correct predictions come from irregular triangles, while shapes with 4–8 sides yield near 0% accuracy, suggesting poor generalization from regular to irregular polygons.

Abstract Shapes: Arrows, stars, and plus signs are common in documents and presentations, thus more likely to appear in pretraining data. GPT-4o and GPT-4 Turbo achieve 84–92% accuracy on these shapes, while open-source models score close to 0%. When these shapes are combined into novel configurations (an arrow on top of a plus sign seen in Figure 1), all models yield 0% accuracy.

The results in Table 5 highlight the influence of pretraining exposure, as models fail to generalize side counting beyond familiar shapes. Instead, their performance confirms our hypothesis that MLLMs rely on System 1 thinking, memorizing shape-side relationships rather than engaging System 2 reasoning to count and interpret visual information. The reliance on memorization over reasoning suggests a need for strategies that can shift MLLMs toward more deliberate, step-by-step problem-solving.

3.4 Visually-Cued CoT

Our results demonstrate that MLLMs heavily rely on System 1 thinking when completing visual-geometry tasks. We explore a potential solution to encourage System 2 reasoning in MLLMs by

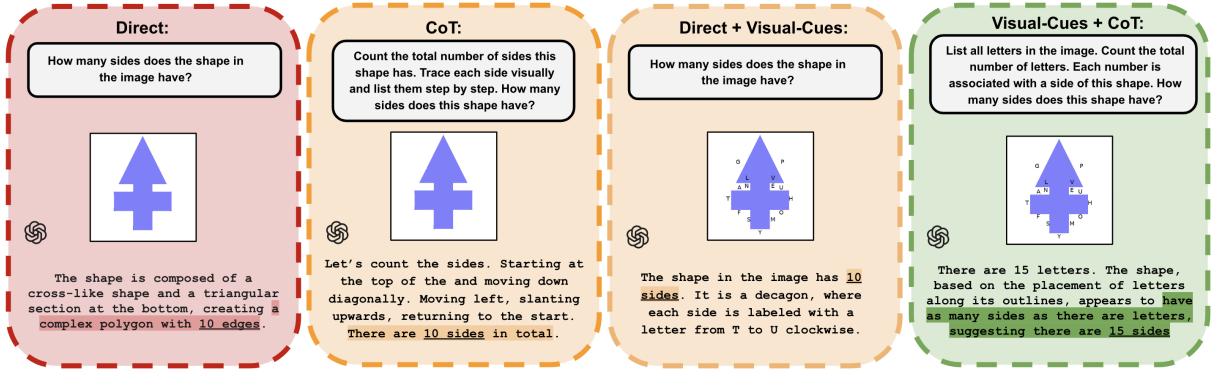


Figure 5: Example outputs from GPT-4-Turbo on “random letters” annotations.

Image Labels	Qwen2-VL	Molmo	Janus-Pro	GPT-4-Turbo	GPT-4o
Heptagon: Direct / VC-CoT					
Numbers 1-7	93% / 100%	60% / 69%	1% / 100%	79% / 98%	43% / 100%
Random Nums	6% / 98%	17% / 29%	3% / 90%	1% / 94%	31% / 100%
Letters A-G	32% / 92%	63% / 75%	15% / 97%	99% / 100%	59% / 100%
Random Letters	0% / 83%	49% / 89%	7% / 74%	7% / 82%	7% / 93%
No Annotations	0% / 99% [†]	1% / 1% [†]	4% / 14% [†]	1% / 3% [†]	22% / 41% [†]
Arrow on Plus: Direct / VC-CoT					
Numbers 1-15	29% / 82%	0% / 61%	13% / 92%	97% / 99%	88% / 99%
Random Nums	0% / 5%	0% / 5%	0% / 0%	1% / 78%	3% / 77%
Letters A-O	0% / 3%	0% / 1%	0% / 2%	75% / 74%	14% / 99%
Random Letters	0% / 0%	0% / 2%	0% / 1%	0% / 25%	3% / 96%
No Annotations	0% / 0% [†]	0% / 0% [†]	0% / 0% [†]	0% / 1% [†]	0% / 1% [†]

Table 6: Performance on heptagons and arrow on plus for Qwen2-VL, Molmo, Janus-Pro, and GPT models. Results are presented as direct / VC-CoT. When there are no image annotations, we use a standard CoT prompt denoted by [†]. VC-CoT is more effective than direct CoT prompting for MLLMs in side counting.

introducing Visually-Cued Chain-of-Thought (VC-CoT) prompting. CoT prompting is widely recognized for improving logical reasoning in LLMs by guiding models to break down complex tasks into intermediate steps (Wei et al., 2023). However, prior work suggests that MLLMs struggle with CoT reasoning (Zhang et al., 2024c,b; Wang et al., 2024a). Existing CoT frameworks for MLLMs focus primarily on the language model without leveraging information available in the image. VC-CoT incorporates explicit “visual cues” into CoT prompts to help bridge the gap between CoT success in LLMs and its effectiveness in MLLMs.

We conduct a case study on two shapes from our previous experiments: a regular heptagon and an arrow on top of a plus sign. For both shapes, models faced significant difficulty in accurately counting their sides, even when the shapes were correctly identified as a “heptagon” or a “triangular shape on a cross”. We consider three conditions: (1) plain images with no annotations, (2) images annotated with ordered numbers or random numbers, and (3) images annotated with ordered letters or

random letters. For each case, we compared direct prompts (e.g., “How many sides does the shape in the image have?”) with CoT prompts that explicitly reference visual annotations (see Figure 5). We use random and ordered letters/numbers when annotating images to ensure that performance is not solely measured under conditions that provide explicit references to the number of sides. Even for plain images with no labels, we included a CoT prompt (seen in Figure 5) to test whether reasoning prompts alone could enhance performance.

Table 6 shows the performance of Qwen2-VL, Molmo, Janus-Pro, GPT-4-Turbo, and GPT-4o, and Appendix Section G shows results for all other models. Molmo and Janus-Pro were explicitly designed for CoT reasoning through dedicated training, while GPT models are well-documented to excel with CoT prompting (Wei et al., 2023), making them the most relevant candidates for VC-CoT. In Table 6, we see that these models exhibit substantial improvements when visually cued CoT prompts are paired with explicit visual annotations, such as numbering or labeling each side. This is especially evident in the GPT models, where GPT-4o increases from 7% to 93%, in the random letters annotation.

Across all models, note that annotations alone or CoT without annotations fail to override visual biases. A striking failure emerges even when models are explicitly given the correct answer through labeled sides (e.g., numbering a heptagon’s sides from 1-7). Non-GPT models still fail without CoT, exposing a fundamental weakness in visual perception. Without VC-CoT prompts, MLLMs fail to extract fine-grained details and default to System 1 memorized associations, hindering structured reasoning. However, linking reasoning steps to visual cues helps mitigate visual biases in these models.

Model	Direct	MathVerse CoT	VC-CoT
Qwen2-VL	35.76%	41.11%	41.95%
Molmo	33.33%	33.33%	38.32%
Janus-Pro	30.60%	29.66%	33.96%
GPT-4-Turbo	45.76%	48.01%	52.41%
GPT-4o	52.12%	55.35%	59.94%
Average	39.51%	41.49%	45.32%

Table 7: **VC-CoT improves performance on MathVerse.** Compared to standard CoT prompting, explicitly guiding models to extract visual information leads to consistent performance improvements.

3.5 Does VC-CoT generalize beyond side-counting?

Given the success of VC-CoT prompts in improving side counting, we evaluate VC-CoT on a complex mathematical reasoning task. Many multimodal geometry datasets already include diagrams annotated with letters, numbers, and angles, providing a natural opportunity to leverage these visual cues. In particular, the vision-dominant split of the MathVerse dataset contains sufficient visual annotations to answer the question. While MathVerse includes CoT prompting, it only slightly modifies the question, changing “please directly answer” to “please first conduct reasoning”, without guiding models to extract the available visual information. In contrast, our visually-cued CoT explicitly prompts models to first identify all present shapes, numbers, and letters and establish their spatial and numerical relationships before answering the question. This structured approach aims to engage System 2 reasoning, forcing models to process the visual structure, retrieve relevant information from the image, and then reason about their relationship.

As shown in Table 7, specifically addressing the visual annotations through VC-CoT improves accuracy across all models. For Molmo and Janus-Pro, MathVerse CoT, does not increase performance, and even decreases compared to the direct prompt. As seen in Appendix Section H, these results generalize to other models as well. While these experiments serve as a case study, they highlight a promising direction for enhancing the reasoning capabilities of MLLMs. We believe incorporating visual cues in CoT prompts can enhance MLLMs’ ability to bridge the gap between vision and language reasoning.

4 Conclusion

Our study highlights fundamental limitations in MLLMs’ ability to integrate visual information for reasoning. While LLM backbones possess strong geometric knowledge, vision encoders remain the primary bottleneck, forcing MLLMs to rely on System 1 thinking instead of System 2 reasoning. Even in simple tasks like shape identification and side counting, MLLMs default to memorized patterns rather than systematically analyzing visual inputs. As an initial step to improving MLLM reasoning, we introduce Visually-Cued Chain-of-Thought (VC-CoT) prompting to engage System 2 reasoning. Our results show that explicitly guiding models to extract and reason about visual cues improves performance, boosting GPT-4o’s accuracy by 86% on our shape dataset as well as enhancing results on MathVerse.

Our findings serve as a broader call to action for the MLLM research community. To effectively pinpoint and address limitations in vision-language integration, we emphasize the need to explore simple, controlled scenarios before engaging with complex multimodal benchmarks. By improving fine-grained visual perception and utilizing visual cues for prompting, researchers can move toward MLLMs that engage in true vision-based System 2 reasoning rather than defaulting to LLM-driven deduction.

5 Limitations

Although we evaluated a diverse range of 13 models, including open-sourced, closed-sourced, and fine-tuned models specialized in mathematical reasoning, there are two recent models we were unable to assess. At the time of writing, the weights for MAVIS (Zhang et al., 2024b) are not publicly available. Additionally, SVE-Math (Zhang et al., 2025), which adapts the vision encoder with a GeoGLIP encoder trained on mathematical diagrams, shows promise for mathematical reasoning. However, there are unresolved issues in the code base, including missing files in the official GitHub repository, which have yet to be addressed. We will add these models to our evaluation suite as soon as these issues are addressed.

Furthermore, our study is designed around a controlled synthetic environment, allowing us to systematically analyze the components of visual geometry datasets. Thus, our method, Visually-Cued Chain-of-Thought (VC-CoT), is particularly geared